

A Divide-and-Conquer Engine for Lexicographical Permutations: Accelerating State Evolution via Hybrid Software-Hardware CPU Instructions

Yusheng HU^{1*}

^{1*}Independent Researcher, Shangdi East Road, Beijing, 100085, China.

Corresponding author(s). E-mail(s): dr.huyusheng@gmail.com;

Abstract

Traditional lexicographical permutation algorithms, epitomized by `std::next_permutation`, are fundamentally bottlenecked by dense control logic and frequent branch mispredictions. This paper introduces LexCHA (Lexicographical Co-designed Hardware Acceleration), a novel hardware-software co-designed architecture that accelerates permutation via native SIMD instructions. Exploiting the inherent fractal isomorphism of permutations, LexCHA decouples global state evolution into a macro software unranking phase and a micro hardware block-construction phase. By utilizing pre-computed deterministic mask streams, LexCHA replaces traditional conditional branching with streaming vector shuffles, achieving a branch-free execution flow. Evaluations demonstrate that LexCHA demonstrates significantly higher throughput than scalar implementations of modern processors, outperforming existing state-of-the-art `std::next_permutation` implementations significantly.

Keywords: Permutation Generation; SIMD Vectorization; Software-Hardware Co-design; Fractal Isomorphism; Combinatorial Computing

MSC Classification: 68Q25 , 05A05

1 Introduction

Lexicographical permutation generation is a fundamental primitive in combinatorial mathematics and algorithm research. However, traditional algorithms, such as the Narayana-Pandita approach [1, 2], struggle on modern superscalar architectures,

creating a critical bottleneck for system throughput. Their reliance on bidirectional scans and nested loops enforces serial execution and frequent conditional branches, which cause significant branch mispredictions and pipeline stalls on deeply pipelined processors.

In the study of permutation generation algorithms, the lexicographic order algorithm is generally considered to be less efficient in terms of both time complexity and implementation performance compared to Heap’s algorithm [3] and the adjacent transposition-based Steinhaus-Johnson-Trotter (SJT) algorithm [6]. Furthermore, the rotation-based methods proposed by Langdon [5] offer an alternative perspective on permutation generation.

In recent years, significant progress has been made in optimizing the performance of permutation generation algorithms. For instance, highly efficient approaches have been developed, including Position Method [7], all of which utilize inverse factorial arrays to drive the generation process. Consequently, improving the generation efficiency of lexicographic algorithms remains a critical objective.

While LexSG [8] established a software framework for branch-free permutation, its throughput remains restricted by sequential execution overhead. To overcome these limitations, this paper proposes LexCHA (Lexicographical Co-designed Hardware Acceleration), a novel hardware-software co-designed architecture. By exploiting the inherent fractal isomorphic topology of lexicographical evolution, LexCHA decouples global state transitions into a dual-layered paradigm and adopts precomputed deterministic execution masks.

At the microarchitectural level, LexCHA implements the core algorithmic logic using native SIMD vector primitives, effectively removing runtime branches and algorithmic redundancy to achieve a streamlined, dataflow-centric execution.

2 Deterministic topological features of lexicographical order evolution

This chapter demonstrates the topological structure of the permutation generation algorithm and reveals that its logical complexity can be transformed into branchless data flow computation through fractal isomorphism.

2.1 Topological deconstruction of permutation space

Traditional lexicographical order generation algorithms rely heavily on the current permutation state to determine subsequent transformations. This dependency causes both the number of available candidate elements and the length of the remaining search path to fluctuate dynamically as the algorithm progresses. This paper models permutation generation as a discrete state traversal on a symmetric group topological space, proving that the evolution between any adjacent states has a deterministic mapping path, providing a theoretical basis for eliminating branching logic.

2.1.1 Definition of the Factorial Number System

Based on the classical approach to permutation encoding introduced by Lehmer [4], we define the structure of the factorial array C to establish a bijective mapping between a permutation and its lexicographical rank:

$$\begin{aligned} C[0] &\in \{0, 1, \dots, n-1\} \\ C[1] &\in \{0, 1, \dots, n-2\} \\ &\vdots \\ C[n-2] &\in \{0, 1\} \\ C[n-1] &\in \{0\} \end{aligned}$$

The traversal of the factorial array follows a standard lexicographical increment, evolving monotonically from the initial state:

$$\mathbf{0} = (0, 0, \dots, 0, 0)$$

to the terminal state:

$$\mathbf{M} = (n-1, n-2, \dots, 1, 0)$$

Table 1 Full Lexicographical State Mapping for $n = 4$

Rank	Fact.	Perm.	Rank	Fact.	Perm.	Rank	Fact.	Perm.
0	0000	1234	8	1100	2314	16	2200	3412
1	0010	1243	9	1110	2341	17	2210	3421
2	0100	1324	10	1200	2413	18	3000	4123
3	0110	1342	11	1210	2431	19	3010	4132
4	0200	1423	12	2000	3124	20	3100	4213
5	0210	1432	13	2010	3142	21	3110	4231
6	1000	2134	14	2100	3214	22	3200	4312
7	1010	2143	15	2110	3241	23	3210	4321

2.1.2 Fractal Block Characteristics of Tail States

By analyzing the state evolution trajectory, we observe that the trailing two elements of the factorial array, $C[n-2 \dots n-1]$, exist in only two valid configurations:

$$(0, 0) \quad \text{and} \quad (1, 0)$$

These configurations manifest in a strict *block-wise clustering* pattern across the state space. Physically, these transitions correspond directly to the pairwise transposition of elements in the underlying permutation, providing the boundary conditions for our SIMD-based acceleration.

2.1.3 Empirical Observation: $n = 4$ Lexicographical Mapping

To visualize this fractal block-wise pattern, we track the state evolution for $n = 4$. As illustrated in Table 2, the periodicity of the trailing 3 elements reveals the clear boundaries suitable for hardware-level vectorization.

Table 2 Full Lexicographical State Evolution and Periodic Block Structure for $n = 4$

Rank	Permutation	Factorial Array (C)	Suffix (D_2, D_1, D_0)	Block ID
0	(0,1,2,3)	(0,0,0,0)	(0,0,0)	Block 0
1	(0,1,3,2)	(0,0,1,0)	(0,1,0)	
2	(0,2,1,3)	(0,1,0,0)	(1,0,0)	
3	(0,2,3,1)	(0,1,1,0)	(1,1,0)	
4	(0,3,1,2)	(0,2,0,0)	(2,0,0)	
5	(0,3,2,1)	(0,2,1,0)	(2,1,0)	
6	(1,0,2,3)	(1,0,0,0)	(0,0,0)	Block 1
7	(1,0,3,2)	(1,0,1,0)	(0,1,0)	
8	(1,2,0,3)	(1,1,0,0)	(1,0,0)	
9	(1,2,3,0)	(1,1,1,0)	(1,1,0)	
10	(1,3,0,2)	(1,2,0,0)	(2,0,0)	
11	(1,3,2,0)	(1,2,1,0)	(2,1,0)	
12	(2,0,1,3)	(2,0,0,0)	(0,0,0)	Block 2
13	(2,0,3,1)	(2,0,1,0)	(0,1,0)	
14	(2,1,0,3)	(2,1,0,0)	(1,0,0)	
15	(2,1,3,0)	(2,1,1,0)	(1,1,0)	
16	(2,3,0,1)	(2,2,0,0)	(2,0,0)	
17	(2,3,1,0)	(2,2,1,0)	(2,1,0)	
18	(3,0,1,2)	(3,0,0,0)	(0,0,0)	Block 3
19	(3,0,2,1)	(3,0,1,0)	(0,1,0)	
20	(3,1,0,2)	(3,1,0,0)	(1,0,0)	
21	(3,1,2,0)	(3,1,1,0)	(1,1,0)	
22	(3,2,0,1)	(3,2,0,0)	(2,0,0)	
23	(3,2,1,0)	(3,2,1,0)	(2,1,0)	

3 Mathematical Foundation: Fractal Block Structure

3.1 Observation of Localized State Evolution

Analyzing the state transitions in the factoradic system (e.g., from range 3000 to 3210), we observe a recurring structure in the trailing elements of the factorial array. Specifically, as the tail of the array evolves from $(0, 0, 0)$ to $(n - k - 1, \dots, 2, 1, 0)$, the corresponding permutations manifest as complete lexicographical sequences of the remaining subset of elements. Regardless of the numerical values of the prefix, the elements within these localized blocks undergo a deterministic sequence of permutations, strictly adhering to lexicographical order. This $k!$ block structure repeats periodically, covering the entire state space of $n!$ permutations.

Observation 1 (Block Lexicographical Completeness) *Given a k -element sub-block of the factorial array ranging from $(0, \dots, 0)$ to $(n - k - 1, \dots, 1, 0)$, the set of permutations generated within this interval constitutes a complete lexicographical sequence of the involved k elements, with a total cardinality of $k!$.*

Observation 2 (Structural Isomorphism) *For any $k!$ sub-block, the logic governing the state transition is invariant. By re-encoding the elements of any k -length prefix to a normalized sequence $\{1, 2, \dots, k\}$, the internal transition dynamics are isomorphic to the lexicographical permutation of the set $\{1, 2, \dots, k\}$.*

3.2 Design Implication: Universal SIMD Acceleration

Building upon Observation 1 and Observation 2, we propose a generalized hardware acceleration logic. Since all $k!$ blocks share the same transformation logic, we can pre-define a universal SIMD permutation kernel. This kernel, once designed for k elements, can be applied to all $n!/k!$ blocks, effectively offloading the entire permutation generation process to a constant-time hardware-accelerated primitive.

4 LexCHA: Fractal-Based SIMD Acceleration

Building upon the topological framework and fractal block properties established in Observation 1, we now translate these theoretical foundations into the concrete hardware acceleration architecture of LexCHA.

4.1 Precomputation of Transition Logic

We pre-calculate the state-transition dynamics for $k = 5$. Each transition is represented as a shuffle mask that transforms the current 128-bit SIMD register state into the subsequent lexicographical state.

Algorithm 1 Precomputation of the SIMD Transition LUT

Require: TAIL_DEPTH = k

```

1: Initialize identity permutation  $P \leftarrow [0, 1, \dots, k - 1]$ 
2: for step  $\leftarrow 0$  to  $k! - 1$  do
3:   Let  $P_{next} \leftarrow \text{next\_permutation}(P)$ 
4:   for  $j \leftarrow 0$  to  $k - 1$  do
5:     flat_lut_N5[step][ $P_{next}[j]$ ]  $\leftarrow P[j]$ 
6:   end for
7:    $P \leftarrow P_{next}$ 
8: end for
```

4.2 Inter-Block Transition and Amortized Analysis

To bridge the transition between consecutive $k!$ sub-blocks, we employ a hybrid control flow. Upon reaching the terminal state of a $k!$ block, the hardware-level SIMD engine triggers an inter-block transition. This is achieved by executing a scalar `next_permutation` on the global n -element sequence.

Observation 3 (Amortized Efficiency) *The generation of permutations relies on the $O(1)$ amortized cost of the standard next-permutation algorithm. Beyond this, our SIMD-accelerated framework further minimizes scalar overhead by processing $k!$ transitions in parallel. Consequently, the inter-block scalar transitions[?]which are inherent to the algorithm’s control flow[?]are effectively diluted, ensuring that the global throughput is primarily bounded by the peak SIMD compute capacity rather than sequential transition overhead.*

This hybrid approach ensures that the performance bottleneck of the `next_permutation` algorithm is minimized. The engine operates in a "macro-jump" fashion, where the scalar logic acts as a coarse-grained supervisor, while the hardware-hardened SIMD primitives provide fine-grained, branch-free execution for the overwhelming majority of permutation states.

5 Overall Architecture Design of LexCHA

5.1 Design Philosophy: Fractal and Divide-and-Conquer

LexCHA aims to reconstruct branch-heavy permutation tasks into branch-free dataflow computation. Lexicographical permutations exhibit inherent fractal properties. We separate the main sequence and data blocks for processing and adopt a hardware-software co-designed divide-and-conquer strategy, as illustrated in Fig. 1. This design fully leverages the Shuffle instructions of modern CPUs. Instead of real-time dynamic suffix scanning, we employ precomputed permutation mappings to trade space overhead for deterministic execution, thereby maximizing hardware utilization.

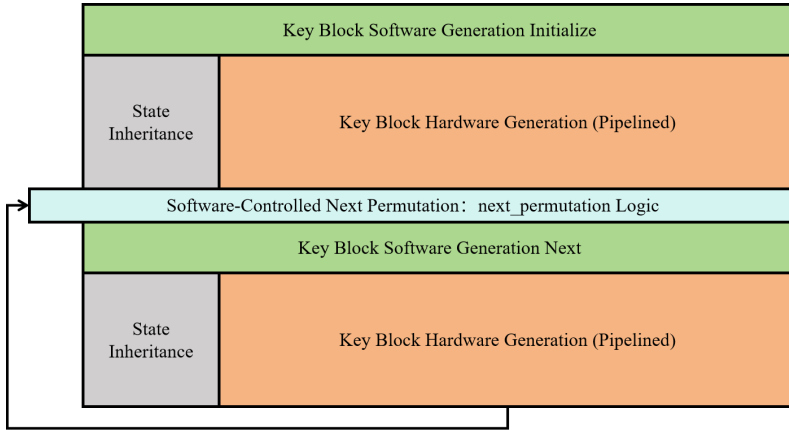


Fig. 1 Co-design architecture of the LexCHA algorithm

5.2 Logic Hardening

Based on vector instruction primitives such as `PSHUFQ` on the x86 architecture, permutation masks are directly loaded into vector registers. All operations are executed

in parallel within the CPU arithmetic logic unit (ALU), achieving physical-level acceleration via hardware-accelerated execution.

We implement a Transition Buffer, whose core component is the Permutation Mask Table (PMT). With the fractal indexing mechanism, PMT only stores basic permutation subsets to reconstruct the evolution logic for permutations of any scale, realizing offline solidification of algorithm logic.

However, a critical trade-off emerges at $K = 5$: the 120 permutations rely on only 10 ($= K(K - 1)/2$) unique transformation masks. While compressing the PMT could save space, it necessitates runtime address indexing and branching. Our experiments show that the overhead of dynamic indexing, specifically index calculation latency and irregular memory access-outweighs the memory footprint benefits. Thus, we utilize a redundant storage strategy for $K = 5$ to ensure a purely sequential execution flow. This approach eliminates runtime jumps, maximizing pipeline efficiency and throughput by maintaining a predictable dataflow.

5.3 Algorithm Pseudocode

Lexicographical permutations have fractal characteristics. We divide the computation into main sequence and sub-blocks, and adopt hardware-software co-designed divide-and-conquer. The proposed design makes full use of modern CPU Shuffle instructions.

Algorithm 2 LexCHA SIMD-Accelerated Permutation Engine

```

1: procedure LEXCHA_ENGINE( $N$ )
2:    $C \leftarrow [0, 1, \dots, N - 1]$  ▷ Initialize sequence
3:    $buffer \leftarrow C[N - TAIL\_DEPTH \dots N - 1]$ 
4:    $p\_reg \leftarrow \text{LoadSIMD}(buffer)$  ▷ Load tail into XMM register
5:    $total\_count \leftarrow 1, max\_perms \leftarrow N!$ 
6:   while  $total\_count < max\_perms$  do
7:     for  $step \leftarrow 0$  to  $FLAT\_STEPS - 1$  do
8:        $mask \leftarrow \text{LoadSIMD}(flat\_lut\_N5[step])$ 
9:        $p\_reg \leftarrow \text{SIMD\_Shuffle}(p\_reg, mask)$  ▷ Hardware-level transition
10:    end for
11:     $total\_count \leftarrow total\_count + FLAT\_STEPS$ 
12:     $buffer \leftarrow \text{StoreSIMD}(p\_reg)$ 
13:     $C[N - TAIL\_DEPTH \dots N - 1] \leftarrow buffer$  ▷ Sync state to memory
14:    if NEXTPERMUTATION( $C$ ) then ▷ Coarse-grained scalar fallback
15:       $total\_count \leftarrow total\_count + 1$ 
16:       $buffer \leftarrow C[N - TAIL\_DEPTH \dots N - 1]$ 
17:       $p\_reg \leftarrow \text{LoadSIMD}(buffer)$  ▷ Reload new state
18:    end if
19:  end while
20:  return  $total\_count$ 
21: end procedure

```

6 LexCHA (Lexicographical Co-designed Hardware Acceleration) Performance Benchmarks

All benchmarks are restricted to lexicographical generation; our approach is specifically optimized for $n \geq 10$ on x86 platforms equipped with SSE/AVX extensions.

6.1 Performance Comparison Test

The Table 3 and Table 4 below present a performance comparison between the standard C++ library implementation (`std::next_permutation`) and the SIMD-accelerated LexCHA engine on various CPU architectures.

- Environment 1: Cloud VM (GitHub Actions / AMD EPYC)
- Compiler: `g++ -O3 -march=native -std=c++17`
- Optimization Target: Strict byte-level vectorization (`_mm_shuffle_epi8`)

Table 3 Performance Comparison: Standard `std::next_permutation` vs. LexCHA Cloud AMD

N	Std (s)	Acc (s)	Std (ns/perm)	Acc (ns/perm)	Speedup
10	0.031685	0.004391	8.73	1.21	7.22x
11	0.357150	0.048634	8.95	1.22	7.34x
12	4.486271	0.589570	9.37	1.23	7.61x
13	60.799209	7.572245	9.76	1.22	8.03x

- Environment 2: Local Host (Intel Core Architecture)
- Compiler: `g++ -O3 -march=native -std=c++17`
- Dedicated Intel shuffle ports and optimal Store-to-Load Forwarding (STLF)

Table 4 Performance Comparison: Standard `std::next_permutation` vs. LexCHA Local Intel

N	Std (s)	Acc (s)	Std (ns/perm)	Acc (ns/perm)	Speedup
10	0.050085	0.003814	13.79	1.05	13.13x
11	0.435035	0.038048	11.77	1.03	11.46x
12	7.151700	0.429768	17.65	1.06	16.60x
13	114.331200	6.793656	18.36	1.09	16.86x

6.2 Throughput Performance Analysis

Experimental results show that the runtime curve of LexCHA remains stable. Its per-permutation latency is nearly unaffected by the permutation length and maintains steady performance.

In contrast, `std::next_permutation` exhibits increasing per-permutation execution time as the value of n increases. It is worth noting that while benchmark results indicate slight variations in optimization depth between AMD and Intel architectures, these differences are primarily attributable to hardware-specific branch prediction and instruction scheduling; they remain orthogonal to the fundamental algorithmic efficiency discussed in this study.

The reference implementation, including the SIMD-optimized routines, is maintained in the *Position-Pure-Algorithm* software repository. This repository provides the benchmark tools necessary to ensure the reproducibility of the performance data presented in Table 3 and Table 4.

7 Conclusion & Future Work

LexCHA thoroughly explores the topological rules behind the evolution of lexicographical permutations, and converts the traditional logic-intensive generation task into SIMD dataflow. This work establishes a hardware-software co-design paradigm of *logic as data, execution as mapping*.

References

- [1] Donald E. Knuth. *The Art of Computer Programming, Volume 4, Fascicle 2: Generating All Tuples and Permutations*. Addison-Wesley Professional, Reading, MA, 2005.
- [2] Robert Sedgewick. Permutation generation methods. *Computing Surveys*, 9:137–164, 1977.
- [3] B. R. Heap. Permutations by interchanges. *The Computer Journal*, 6(3):293–294, 1963.
- [4] D. H. Lehmer. Teaching combinatorial tricks to a computer. *Proc. Sympos. Appl. Math.*, 10:179–193, 1960.
- [5] G. G. Langdon. Rotation-based permutation generation methods. *IBM Journal of Research and Development*, 17(5):420–425, 1973.
- [6] S. M. Johnson. “Generation of permutations by adjacent transpositions,” *Mathematics of Computation*, vol. 17, no. 83, pp. 282–285, 1963.
- [7] Yusheng Hu. Position Method: A Linear-Time Generation Algorithm for Permutations. *SSRN*, February 22, 2026. Available at <https://ssrn.com/abstract=6285581> or <http://dx.doi.org/10.2139/ssrn.6285581>.
- [8] Yusheng Hu, “LexSG: Accelerating Lexicographical Permutations through Factorial-Based Structure Generation,” arXiv preprint arXiv:XXXX.XXXXX, 2026 (Submitted).